2 Number Systems and Codes

2.1. INTRODUCTION

A digital computer is also known as *data processor.* It processes data while solving a mathematical problem or doing translation from one language to another etc. Before a computer can process data, the data has to be converted into a form acceptable to a computer.

We use decimal number system in our work. This system has digits 0, 1, 2, 3, 4, 5, 6, 7, 8 and 9. Computers cannot use these numbers. Instead, a computer works on binary digits. A binary number system has only two digits 0 and 1. This is because of the reason that computers use integrated circuits with thousands of transistors. Due to variation of parameters the behaviour of transistor can be very erratic and quiescent point may shift from one position to another. Nevertheless the cut off and saturation points are fixed. When a transistor is cut off, a large change in values is needed to change the state to saturation. Similar is the situation when it is in saturation. Thus a transistor is a very reliable two state device. One state represents digit 0 and the other state represents digit 1. All input voltages are recognised as either 0 or 1.

2.2. TWO STATE DEVICES

A switch can either be in off position or on position. Thus it is a two state device*. One position means 0 and the other position means 1. A magnetic core of ferrite has a rectangular hysteresis loop shown in Fig. 2.1. The retentivity is very high. Even if the magnetising current is reduced to zero, the flux remains at the same value. Thus a ferrite core can exist only in two states unmagnetised and magnetised which may represent 0 and 1.

A group of many two state devices is known as a register. Each device can be either in state 0 or 1. Thus the group will depict a string of zeros and ones.

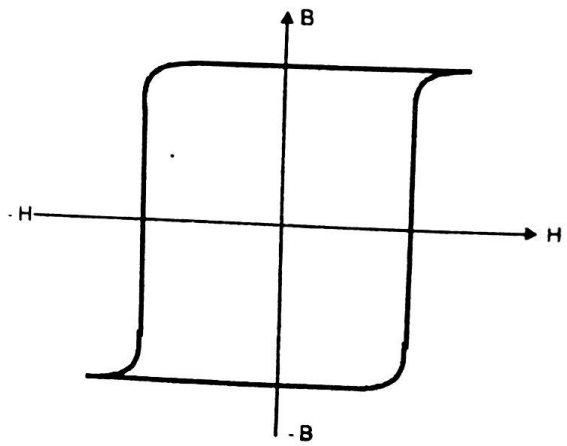

Fig. 2.1. B-H curve of Ferrite core

Fig. 2.2 shows four transistors $T_1 - T_4$. The base voltages of T_1 and T_2 are 10 V each and both these transistors are conducting (in saturation state) and the output voltage is 0 each. The transistors T_3 and T_4 have base voltage zero and are cut off. The output voltage of T_3 and T_4 is 10 V. The output voltages of T_1 and T_2 are low (representing digit 0) and those of T_3 and T_4 are high (representing digit 1). Thus this group of transistors represents the string 0011.

2.3. DECIMAL NUMBER SYSTEM

We are all familiar with the decimal number system. It uses ten digits (0, 1, 2, 3, 4, 5, 6, 7, 8, 9) and thus its base is 10. The decimal number system of counting was evolved because we have 8 fingers and 2 thumbs on our two hands so that we can count 10. By using the different digits in different positions we

* Some examples of two state devices are : lights (light on represents 1 and light off represents 0), punched card (hole represents 1 and absence of hole represents 0), magnetic tape (a magnetised point in the tape represents 1 while an unmagnetised point represents 0).

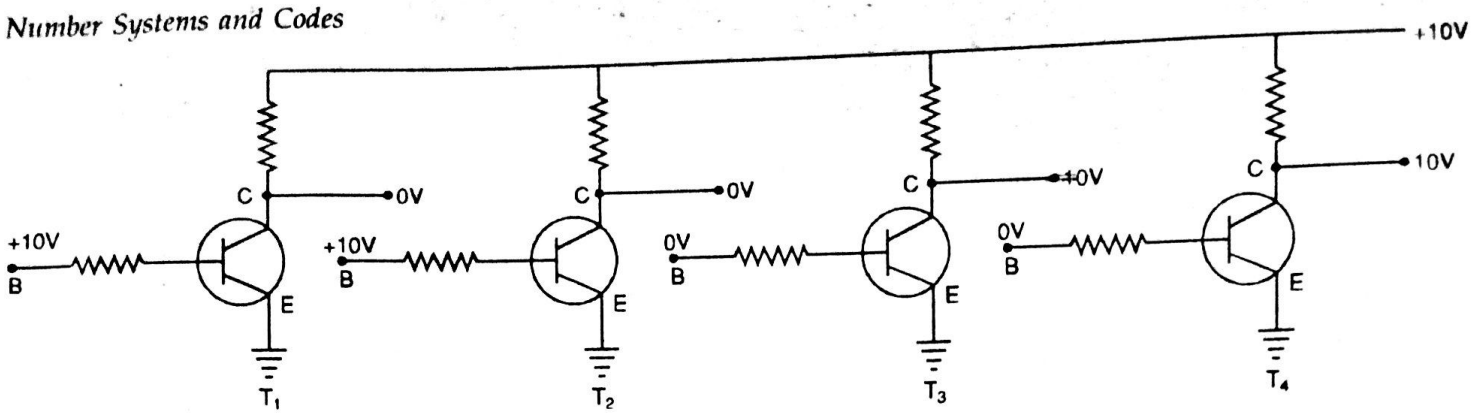

Fig. 2.2. Transistor register with contents 0011

can express any number. For numbers bigger than 9 we use two or more digits. The position of each digit in the number indicates the magnitude that this number represents. In the number 27, the digit 7 represents 7×10^0 or 7 and the digit 2 represents 2×10^1 or 20. The sum of 7 and 20 makes 27. Similarly the number 263 can be expressed as

Decimal $263 = (2 \times 10^2) + (6 \times 10^1) + (3 \times 10^0) = 200 + 60 + 3 = 263$

Since the base in decimal number system is 10, the number 263 can be written as 263_{10}. The suffix 10 emphasizes the fact that the base is 10.

2.4. BINARY NUMBER SYSTEM

The binary number system has only two digits 0 and 1. Thus a binary number is a string of zeros and ones. Since it has only two digits, the base is 2.

The abbreviation of binary digit is bit. The binary number 1100 has 4 bits, 101011 has 6 bits and 11001010 has 8 bits. Each bit may represent either 0 or 1. A string of 8 bits is known as a byte. A byte is the basic unit of data in computers. In most computers, the data* is processed in strings of 8 bits or some multiples (*i.e.*, 16, 24, 32 etc.). The computer memory also stores data in strings of 8 bits or multiples of 8 bits. Table 2.1 shows the 16 combinations of a 4 bit binary word.

Table 2.1. Binary, decimal, hexadecimal and octal equivalence

Binary				Decimal	Hexadecimal	Octal
0	0	0	0	0	0	0
0	0	0	1	1	1	1
0	0	1	0	2	2	2
0	0	1	1	3	3	3
0	1	0	0	4	4	4
0	1	0	1	5	5	5
0	1	1	0	6	6	6
0	1	1	1	7	7	7
1	0	0	0	8	8	10
1	0	0	1	9	9	11
1	0	1	0	10	A	12
1	0	1	1	11	B	13
1	1	0	0	12	C	14
1	1	0	1	13	D	15
1	1	1	0	14	E	16
1	1	1	1	15	F	17

It is interesting to note that our earlier system of counting and weighing was basically a binary system.Two Annas make one Two Annas. Two two annas make one Chawani (quarter of a rupee) and so on. Similary two Chhattaks (one sixteenth of seer) make one aad pao (quarter seer). Two aad pao make one pao (quarter seer) and so on.

* Data means the names, numbers etc. needed to solve a problem.

As in the decimal system the binary number system is positionally weighted. The digit on the extreme right hand side has a weight of 2^0, the next one has a weight of 2^1 and so on. Since the base is 2, the binary number is written as (say) 1011_2. The suffix 2 emphasizes the fact that base is 2. If the number of binary digits is n, the highest decimal number which can be counted is $2^n - 1$. Thus with 4 binary digits we can count upto $(2^4 - 1)$ or 15 decimal number. If $n = 8$ we can count upto $(2^8 - 1) = 255$ decimal number.

2.4.1. Binary to Decimal Conversion

The procedure to convert a binary number to decimal is called dibble–dabble method. We start with the left hand bit. Multiply this value by 2 and add the next bit. Again multiply by 2 and add the next bit. Stop when the bit on extreme right hand side is reached.

An other fast and easy method to convert binary number to decimal number is as under :

1. Write the binary number.
2. Write the weights $2^0, 2^1, 2^2, 2^3$ etc., under the binary digits starting with the bit on right hand side.
3. Cross out weights under zeros.
4. Add the remaining weights.

Example 2.1. Convert 100101_2 to decimal.

Solution : Left hand bit 1

Multiply by 2 and add next bit $2 \times 1 + 0 =$ 2

Multiply by 2 and add next bit $2 \times 2 + 0 =$ 4

Multiply by 2 and add next bit $2 \times 4 + 1 =$ 9

Multiply by 2 and add next bit $2 \times 9 + 0 =$ 18

Multiply by 2 and add next bit $2 \times 18 + 1 = 37$

Therefore, $100101_2 = 37_{10}$

Example 2.2. Convert 1101_2 into equivalent decimal number.

Solution :

1	1	0	1	Binary number
8	4	2	1	Write weights
8	4	~~2~~	1	Cross out weights under zeros
$8 + 4 + 0 + 1 = 13$				Add weights

Therefore, $1101_2 = 13_{10}$

Example 2.3. Convert 110011011001_2 into equivalent decimal number.

Solution :

1	1	0	0	1	1	0	1	1	0	0	1	Binary number
2048	1024	512	256	128	64	32	16	8	4	2	1	Write weights
2048	1024	~~512~~	~~256~~	128	64	~~32~~	16	8	~~4~~	~~2~~	1	Cross out weights under zeros

$2048 + 1024 + 128 + 64 + 16 + 8 + 1 = 3289$ Add weights

Thus $110011011001_2 = 3289_{10}$

2.4.2. Decimal to Binary Conversion

A systematic way to convert a decimal number into equivalent binary number is known as double dabble. This method involves successive division by 2 and recording the remainder (the remainder will be always 0 or 1). The division is stopped when we get a quotient of 0 with a remainder of 1. The remainders when read upwards give the equivalent binary number.

Example 2.4. Convert decimal number 10 into its equivalent binary number.

Solution :

2	10	
2	5	remainder 0
2	2	remainder 1
2	1	remainder 0
	0	remainder 1

The binary number is 1010.

Example 2.5. Convert decimal number 25 into its binary equivalent.

Solution :

2	25	
2	12	remainder 1
2	6	remainder 0
2	3	remainder 0
2	1	remainder 1
	0	remainder 1

The binary number is 11001.

Example 2.6. Convert 3289_{10} into binary number and check the result by comparing with example 2.3.

Solution :

2	3289	
2	1644	remainder 1
2	822	remainder 0
2	411	remainder 0
2	205	remainder 1
2	102	remainder 1
2	51	remainder 0
2	25	remainder 1
2	12	remainder 1
2	6	remainder 0
2	3	remainder 0
2	1	remainder 1
	0	remainder 1

Therefore, $3289_{10} = 110011011001_2$

The answer is the same as example 2.3.

2.5. BINARY ARITHMETIC

2.5.1. Binary Addition

The rules for addition of binary numbers are as under :

$0 + 0 = 0$

$0 + 1 = 1 + 0 = 1$

$1 + 1 = 10$ *i.e.*, 1 + 1 equals 0 with a carry of 1 to next higher column

$1 + 1 + 1 = 11$ *i.e.*, 1 + 1 + 1 equals 1 with a carry of 1 to next higher column.

2.5.2. Binary Subtraction

The rules for subtraction of binary numbers are as under :

$0 - 0 = 0$

$1 - 0 = 1$

$1 - 1 = 0$

$10 - 1 = 1$

In both the operations of addition and subtraction, we start with the least significant bit (L.S.B.), i.e the bit on the extreme right hand side and proceed to the left (as is done in decimal addition and subtraction).

Example 2.7. *(a)* Convert decimal numbers 15 and 31 into binary numbers. *(b)* Add the binary numbers and convert the result into decimal equivalent.

Solution : *(a)*

2	15	
2	7	remainder 1
2	3	remainder 1
2	1	remainder 1
	0	remainder 1

Binary number is 1111

2	31	
2	15	remainder 1
2	7	remainder 1
2	3	remainder 1
2	1	remainder 1
	0	remainder 1

Binary number is 11111

(b) Binary addition

$$\begin{array}{cccccc} & 1 & 1 & 1 & 1 & \\ 1 & 1 & 1 & 1 & 1 & \\ \hline 1 & 0 & 1 & 1 & 1 & 0 \end{array}$$

The sum of binary numbers 1111 and 11111 is the binary number 101110

1	0	1	1	1	0	Binary number
32	16	8	4	2	1	Write weights
32	~~16~~	8	4	2	~~1~~	Cross out weights under zero

$32 + 8 + 4 + 2 = 46$ Add weights

Example 2.8. Subtract 10001 from 11001.

Solution :

$$\begin{array}{cccccc} & 1 & 1 & 0 & 0 & 1 \\ (-) & 1 & 0 & 0 & 0 & 1 \\ \hline & 0 & 1 & 0 & 0 & 0 \end{array} \qquad \begin{array}{r} 25 \\ (-)\ 17 \\ \hline 8 \end{array}$$

Example 2.9. Subtract 0111 from 1010.

Solution :

$$\begin{array}{cccccc} & 1 & 0 & 1 & 0 & \\ (-) & 0 & 1 & 1 & 1 & \\ \hline & 0 & 0 & 1 & 1 & \end{array} \qquad \begin{array}{r} 10 \\ (-)\ 7 \\ \hline 3 \end{array}$$

The least significant digit* in the first number is 0. So we borrow 1 from the next digit and subtract 1 to give 1. Now in the second column we have 0, so we again borrow 1 from the next higher column and subtract 1 to give 1. In the third column, we borrow 1 from the next higher column and 1 – 1 gives 0. In the fourth column, 0 (after lending) – 0 gives 0.

* The digit on the extreme RHS is the LSB and the digit on extreme LHS is the MSB (most significant digit)

2.5.3. Binary Multiplication

The four basic rules for binary multiplication are :

$$0 \times 0 = 0$$
$$0 \times 1 = 0$$
$$1 \times 0 = 0$$
$$1 \times 1 = 1$$

The method of binary multiplication is similar to that in decimal multiplication. The method involves forming partial products, shifting successive partial products left one place and then adding all the partial products.

Example 2.10. *(a)* Multiply 1011_2 by 101_2.

(b) Convert 1011_2 and 101_2 into decimal numbers. Multiply them, convert the result into binary and compare with the result of part *(a)*.

Solution : *(a)*

$$\begin{array}{r} 1\ 0\ 1\ 1 \\ \times\ 1\ 0\ 1 \\ \hline 1\ 0\ 1\ 1 \\ 0\ 0\ 0\ 0 \\ 1\ 0\ 1\ 1 \\ \hline 1\ 1\ 0\ 1\ 1\ 1 \end{array}$$

(b)

1 0 1 1	Binary number
8 4 2 1	Write weights
8 ~~4~~ 2 1	Cross out weights under zero
8 + 2 + 1 = 11	Add weights
1 0 1	Binary number
4 2 1	Write weights
4 ~~2~~ 1	Cross out weights under zero
4 + 1 = 5	Add weights

$11 \times 5 = 55$

2	55	
2	27	remainder 1
2	13	remainder 1
2	6	remainder 1
2	3	remainder 0
2	1	remainder 1
	0	remainder 1

$55_{10} = 110111_2$

The result is the same as in part *(a)*.

Example 2.11. Multiply 11101_2 by 10001_2.

Solution :

$$\begin{array}{r} 1\ 1\ 1\ 0\ 1 \\ \times\ 1\ 0\ 0\ 0\ 1 \\ \hline 1\ 1\ 1\ 0\ 1 \\ 0\ 0\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0\ 0 \\ 0\ 0\ 0\ 0\ 0 \\ 1\ 1\ 1\ 0\ 1 \\ \hline 1\ 1\ 1\ 1\ 0\ 1\ 1\ 0\ 1 \end{array}$$

The reader should convert 11101_2 and 10001_2 into decimal numbers, multiply them, convert the result into equivalent binary number and compare with the above result.

2.5.4. Binary Division

The division in binary system follows the same long division procedure as in decimal system.

Example 2.12. *(a)* Divide 110110_2 by 101.

(b) Convert 110110_2 and 101_2 into equivalent decimal number obtain division, convert results into binary and compare the results with those in part *(a)*.

Solution : *(a)*

$$\begin{array}{r} 1\ 0\ 1\ 0 \\ 101\ \overline{)\ 1\ 1\ 0\ 1\ 1\ 0\ (} \\ 1\ 0\ 1 \\ \hline 1\ 1\ 1 \\ 1\ 0\ 1 \\ \hline 1\ 0\ 0 \end{array}$$

1010 quotient

remainder

(b)

1	1	0	1	1	0	Binary number
32	16	8	4	2	1	Write weights
32	16	~~8~~	4	2	~~1~~	Cross out weights under zero

32 + 16 + 4 + 2 = 54 — Add weights

1	0	1	Binary number
4	2	1	Write weights
4	~~2~~	1	Cross out weights under zero

4 + 1 = 5 — Add weights

$$\begin{array}{r|l} 5 & 54 \\ \hline & 10-4 \end{array}$$

quotient remainder

2 |	10	
2 |	5	remainder 0
2 |	2	remainder 1
2 |	1	remainder 0
	0	remainder 1

2 |	4	
2 |	2	remainder 0
2 |	1	remainder 0
	0	remainder 1

The quotient is 1010_2 and the remainder is 100_2. These are the same as in part *(a)*.

Example 2.13. Divide 10100111_2 by 1001_2.

Solution :

$$\begin{array}{r} 1\ 0\ 0\ 1\ 0 \\ 1001\ \overline{)\ 1\ 0\ 1\ 0\ 0\ 1\ 1\ 1\ (} \\ 1\ 0\ 0\ 1 \\ \hline 1\ 0\ 1\ 1 \\ 1\ 0\ 0\ 1 \\ \hline 1\ 0\ 1 \end{array}$$

10010 quotient

remainder

The reader should convert 10100111_2 and 1001_2 into decimal numbers, obtain division, convert the results of quotient and remainder into binary equivalent and compare with the above results.

2.6. SIGNED BINARY NUMBERS

To represent negative numbers in the binary system, digit 0 is used for the + sign and 1 for the – sign. The most significant bit is the sign bit followed by the magnitude bits. Numbers expressed in this manner are known as signed binary numbers*. The numbers may be written in 4 bits, 8 bits, 16 bits, etc. In every case, the leading bit represents the sign and the remaining bits represent the magnitude.

Example 2.14. Express in 16–bit signed binary system : *(a)* + 8, *(b)* – 8, *(c)* 165, *(d)* – 165.

Solution : *(a)*

2	8	
2	4	remainder 0
2	2	remainder 0
2	1	remainder 0
	0	remainder 1

The binary number is 1000.

For the 16 bit system, we use 16 bits, 0 (which stands for +) in the leading position, 1000 in the last 4 bits and 0 in the remaining 11 positions. So the signed 16 bit binary number is

+ 8 = 0000 0000 0000 1000

(b) In the leading bit we will have 1 (to represent the '–' sign). The rest of the representation is the same as in part *(a)*.

– 8 = 1000 0000 0000 1000

(c)

2	165	
2	82	remainder 1
2	41	remainder 0
2	20	remainder 1
2	10	remainder 0
2	5	remainder 0
2	2	remainder 1
2	1	remainder 0
	0	remainder 1

So the number is 10100101.

Using 0 in the leading bit (for + sign) the 16 bit signed binary number is

+ 165 = 0000 0000 1010 0101

(d) In the leading bit position we will have 1 (for the '–' sign).

Therefore, – 165 = 1000 0000 1010 0101

2.7. 1'S COMPLEMENT

The 1's complement of a binary number is obtained by complementing each bit (*i.e.*, 0 for 1 and 1 for 0).

Thus each bit in the original word is inverted to give the 1's complement. For example, for the number

1 1 0 0 1 0 0 1

1's complement is 0 0 1 1 0 1 1 0

2.8. 2'S COMPLEMENT

The signed binary numbers required too much electronic circuitry for addition and subtraction. Therefore, positive decimal numbers are expressed in sign–magnitude form but negative decimal numbers are expressed in 2's complements.

2's complement is defined as the new word obtained by adding 1 to 1's complement*, *e.g.*,

Let	A	=	0	1	0	1	*i.e.*, 5
1's complement	$\overline{A}$	=	1	0	1	0	
		+				1	
2's complement	A'	=	1	0	1	1	*i.e.*, – 5

Taking the 2's complement is the same as changing the sign of the given binary number. If we take the 2's complement twice we get the original number, *e.g.*,

	A'	=	1	0	1	1	
1's complement	$\overline{A'}$	=	0	1	0	0	
		+				1	
2's complement	A''	=	0	1	0	1	$= A$

Thus $A'' = A$. In view of this every number and its 2's complement form a complementary pair. In a typical computer positive numbers are expressed in sign magnitude form but negative numbers are expressed as 2's complements. The positive numbers have a leading sign bit of 0 and negative numbers have a leading sign bit of 1.

Example 2.15. What do the following represent in 2's complement representation ?

(a) 0 0 0 1 1 1 1 1 *(b)* 1 1 1 0 0 1 0 1 *(c)* 1 1 1 1 0 1 1 1.

Solution : *(a)*

0	0	0	1	1	1	1	1	Binary number
	64	32	16	8	4	2	1	Write weights
	~~0~~	~~0~~	16	8	4	2	1	Cross out weights under 0
				16 + 8 + 4 + 2 + 1 = 31				Add weights

Thus, the number is + 31

(b)

A	=	1	1	1	0	0	1	0	1
$\overline{A}$	=	0	0	0	1	1	0	1	0
	+								1
A'	=	0	0	0	1	1	0	1	1
					16	8	4	2	1
					16	8	~~0~~	2	1
					16 + 8 + 2 + 1 = 27				

Write weights
Cross out weights under 0
Add weights

Thus, $A' = 27$. Therefore, $A = -27$

(c)

A	=	1	1	1	1	0	1	1	1
$\overline{A}$	=	0	0	0	0	1	0	0	0
	+								1
A'	=	0	0	0	0	1	0	0	1
						8	4	2	1
						8	~~0~~	~~0~~	1
						8 + 1 = 9			

Write weights
Cross out weights under 0
Add weights

Thus, $A' = 9$. Therefore, $A = -9$

* 1's complement of A is denoted by $\overline{A}$ and 2's complement of A is denoted by A'.

Example 2.16. Find the largest positive and negative numbers which can be stored with 8 bits.

Solution : The largest positive number is

0 1 1 1 1 1 1 1 = + 127

The largest negative number is

1 0 0 0 0 0 0 0 = – 128

Thus, with 8 bits we can store numbers between – 128 and + 127.

2.9. 2'S COMPLEMENT ADDITION, SUBTRACTION

The use of 2's complement representation has simplified the computer hardware* for arithmetic operations. When A and B are to be added, the B bits are not inverted so that we get,

$$S = A + B \qquad ...(2.1)$$

When B is to be subtracted from A, the computer hardware forms the 2's complement of B and then adds it to A. Thus

$$S = A + B' = A + (-B) = A - B \qquad ...(2.2)$$

Eqns. (2.1 and 2.2) represent algebraic addition and subtraction. A and B may represent either positive or negative numbers. Moreover, the final carry has no significance and is not used.

Example 2.17. If $A = -24$ and $B = +16$, *(a)* represent A and B in 8–bit 2's complement, *(b)* find $A + B$, *(c)* find $A - B$.

Solution : *(a)*

2	24	
2	12	remainder 0
2	6	remainder 0
2	3	remainder 0
2	1	remainder 1
	0	remainder 1

2	16	
2	8	remainder 0
2	4	remainder 0
2	2	remainder 0
2	1	remainder 0
	0	remainder 1

$\underline{24} = 0\ 0\ 0\ 1\ 1\ 0\ 0\ 0$

$24 = 1\ 1\ 1\ 0\ 0\ 1\ 1\ 1$

$+ \qquad\qquad\qquad\qquad 1$

$-24 = 1\ 1\ 1\ 0\ 1\ 0\ 0\ 0$

$B = 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0$

(b)

$-24 = 1\ 1\ 1\ 0\ 1\ 0\ 0\ 0$

$+16 = 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0$

$-8 = 1\ 1\ 1\ 1\ 1\ 0\ 0\ 0$

The number 1 1 1 1 1 0 0 0 can be converted into a decimal to check that the result is –8.

(c)

$B = 0\ 0\ 0\ 1\ 0\ 0\ 0\ 0$

$\overline{B} = 1\ 1\ 1\ 0\ 1\ 1\ 1\ 1$

$+ \qquad\qquad\qquad\qquad 1$

$B' = 1\ 1\ 1\ 1\ 0\ 0\ 0\ 0 = -16$

$-24 = 1\ 1\ 1\ 0\ 1\ 0\ 0\ 0$

$+(-16) = 1\ 1\ 1\ 1\ 0\ 0\ 0\ 0$

$-40 = 1\ 1\ 0\ 1\ 1\ 0\ 0\ 0$

(Convert 1 1 0 1 1 0 0 0 to decimal and check that it equals –40).

* Hardware means electronic, mechanical and magnetic devices in a computer. The computer program is known as software

Example 2.18. If $A = -60$ and $B = -28$, *(a)* represent A and B in 8 bit 2's complement, *(b)* find A, *(c)* find $B - A$.

Solution : *(a)*

28	=	0	0	0	1	1	1	0	0
$\overline{28}$	=	1	1	1	0	0	0	1	1
	+								1
−28		1	1	1	0	0	1	0	0

60	=	0	0	1	1	1	1	0	0
$\overline{60}$	=	1	1	0	0	0	0	1	1
	+								1
−60		1	1	0	0	0	1	0	0

(b)

−28	1	1	1	0	0	1	0	0
−60	1	1	0	0	0	1	0	0
−88	= 1	0	1	0	1	0	0	0

(Neglect final carry of 1)

(Convert 1 0 1 0 1 0 0 0 to decimal and check that it equals −88)

(c) 2's complement of −60 is +60 = 0 0 1 1 1 1 0 0

−28	1	1	1	0	0	1	0	0
−(−60) = +60	0	0	1	1	1	1	0	0
+32	0	0	1	0	0	0	0	0

(Neglect final carry of 1)

(Convert 0 0 1 0 0 0 0 0 to decimal and check that it equals +32)

2.10. BINARY FRACTIONS

So far we have discussed only whole numbers. However, to represent fractions is also importa
The decimal number 2568 is represented as

$$2568 = 2000 + 500 + 60 + 8 = 2 \times 10^3 + 5 \times 10^2 + 6 \times 10^1 + 8 \times 10^0$$

Similarly, 25.68 can be represented as

$$25.68 = 20 + 5 + 0.6 + 0.08 = 2 \times 10^1 + 5 \times 10^0 + 6 \times 10^{-1} + 8 \times 10^{-2}$$

2.10.1. Conversion of Binary to Decimal

In the binary system, the weights of the binary bits after the binary point, can be written as

$$0.1011 = 1 \times 2^{-1} + 0 \times 2^{-2} + 1 \times 2^{-3} + 1 \times 2^{-4}$$

$$= 1 \times \frac{1}{2} + 0 \times \frac{1}{4} + 1 \times \frac{1}{8} + 1 \times \frac{1}{16}$$

$$= 0.5 + 0 + 0.125 + 0.0625 = 0.6875 \text{ (decimal)}$$

2.10.2. Conversion of Decimal to Binary

To convert a decimal to binary, a method of successive multiplication by 2 is used. After ea multiplication, the integer part is noted separately and the fraction is again multiplied by 2 till the remaind becomes zero. Sometimes it is possible that the remainder does not become zero even after many stage In such a case, an approximation is made and the result is taken upto a certain number of bits after th binary point. A similar procedure is adopted for a number having both integer and fractions. Binary fractio can be added, subtracted etc., as the decimal numbers.

Example 2.19. Express the number 0.6875 into binary equivalent.

Solution :

Fraction	Fraction × 2	Remainder new fraction	Integer
0.6875	1.375	0.375	1 (MSB)
0.375	0.75	0.75	0
0.75	1.5	0.5	1
0.5	1	0	1 (LSB)

The binary equivalent is 0.1011.

Example 2.20. Convert the decimal number 0.634 into its binary equivalent.

Solution :

Fraction	Fraction × 2	Remainder new fraction	Integer
0.634	1.268	0.268	1 (MSB)
0.268	0.536	0.536	0
0.536	1.072	0.072	1
0.072	0.144	0.144	0
0.144	0.288	0.288	0
0.288	0.576	0.576	0
0.576	1.152	0.152	1 (LSB)

It is seen that it is not possible to get a zero as remainder even after 7 stages. The process can be continued further or an approximation can be made and the process terminated here.

The binary equivalent is 0.1010001. It is important to note that the decimal equivalent of 0.1010001 (binary) is 0.6328125 (decimal).

Example 2.21. Convert the decimal number 39.12 into binary.

Solution : Taking the integer part first

2	39	
2	19	remainder 1 (LSB)
2	9	remainder 1
2	4	remainder 1
2	2	remainder 0
	1	remainder 0
	0	remainder 1 (MSB)

The result is 100111

Taking the fractional part

Fraction	Fraction × 2	Remainder new fraction	Integer
0.12	0.24	0.24	0 (MSB)
0.24	0.48	0.48	0
0.48	0.96	0.96	0
0.96	1.92	0.92	1
0.92	1.84	0.84	1
0.84	1.68	0.68	1
0.68	1.36	0.36	1 (LSB)

This fraction cannot be converted into exact binary number. The process can be terminated here. The result is 0.0001111.

Adding the binary equivalent of 39 and 0.12

$$\begin{array}{r} 1\,0\,0\,1\,1\,1 \;.\; 0\,0\,0\,0\,0\,0\,0 \\ 0 \;.\; 0\,0\,0\,1\,1\,1\,1 \\ \hline 1\,0.\,0\,1\,1\,1 \;.\; 0\,0\,0\,1\,1\,1\,1 \end{array}$$

Example 2.22. *(a)* Multiply 10.1_2 and 1.01_2. Convert the result into equivalent decimal number.

(b) Convert 10.1_2 and 1.01_2 into decimal numbers and multiply the decimal numbers. Compare with the result of part *(a)*.

Solution : *(a)*

```
      1 0 . 1
        1 . 0 1
    -------------
      1 0 1
    0 0 0
  1 0 1
-----------------
1 1 . 0 0 1
```

1 1 .	0 0 1	Binary number
$2^1\ 2^0$.	$2^{-1}\ 2^{-2}\ 2^{-3}$	Write weights
$2^1\ 2^0$.	$\not{2}^{-1}\ \not{2}^{-2}\ 2^{-3}$	Cross out weight under zero
3.125		Add weights

$11.001_2 = 3.125_{10}$

(b) $10.1_2 = 2.5_{10}$

$1.01_2 = 1.25_{10}$

$2.5_{10} \times 1.25_{10} = 3.125_{10}$

Example 2.23. Mulitply 110.11_2 and 110_2 and convert the result into equivalent decimal numbe...

Solution :

```
        1 1 0 . 1 1
          1 1 0
    -------------
        0 0 0 0 0
      1 1 0 1 1
    1 1 0 1 1
-----------------
1 0 1 0 0 0 . 1 0
```

1 0 1 0 0 0 .	1 0	Binary number
$2^5\ 2^4\ 2^3\ 2^2\ 2^1\ 2^0$.	$2^{-1}\ 2^{-2}$	Write weights
$2^5\ \not{2}^4\ 2^3\ \not{2}^2\ \not{2}^1\ \not{2}^0$.	$2^{-1}\ \not{2}^{-2}$	Cross out weight under zero
32 + 8 + 0.5 = 40.5		Add weights

$101000.10_2 = 40.5_{10}$

The reader should convert 110.11_2 and 110_2 into equivalent decimal numbers, multiply them and s... that the result is 40.5.

Example 2.24. *(a)* Divide 111.10_2 by $.101_2$.

(b) Divide 10.1101_2 by 1.01_2.

Solution : *(a)*

```
.101 ) 1 1 1 . 1 0 ( 1100
       1 0 1
       -----
         1 0 1
         1 0 1
         -----
           0 0
```

(b)

```
1.01 ) 1 0 . 1 1 0 1 ( 10.01
       1 0 1
       -----
         1 0 1
         1 0 1
         -----
           0 0
```

The reader should convert the numbers into equivalent decimal numbers, obtain the division an... compare the results.

2.11. DOUBLE PRECISION NUMBERS

Most present day computers are 16 bit. In these computers the numbers from + 32,767 to – 32,76... can be stored in each register. To store numbers greater than these numbers, double precision syste... is used. In this method two storage locations are used to represent each number. The format is

First word	S	High order bits
Second word	O	Low order bits

S is the sign bit and *O* is a zero. Thus numbers with 31 bit length can be represented in 16 bit registers. For still bigger numbers triple precision can be used. In triple precision 3 word lengths (each 16 bit) is use... to represent each number.

2.12. FLOATING POINT NUMBERS

Most of the time we use very small and very large numbers, *e.g.*, 1.02×10^{-12} and $6.5 \times 10^{+17}$. In binar... representation, the numbers are expressed by using a mantissa and an exponent.

The mantissa has a 10 bit length and exponent has 6 bit length. Fig. 2.3 shows one such representation.

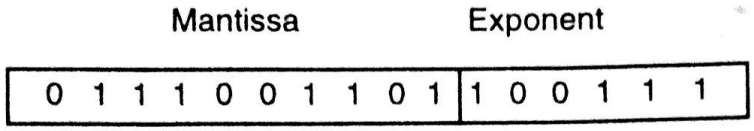

Fig. 2.3. Floating point format

The lert most bit of mantissa is sign bit. The binary point is to the right of this sign bit.

The 6 bit exponent has a base of 2. The exponent can represent numbers 0 to 63. To express negative exponents the number 32_{10} (*i.e.*, 100000_2) has been added to the exponent. It is known as excess–32 notation and is a common floating point format. Examples of exponent in excess–32 format are :

Actual exponent	**Binary representation in excess–32 format**
–32	000000
–1	011111
0	100000
+7	100111
+15	101111
+31	111111

The number represented in Fig. 2.3 is

Mantissa +0.111001101

Exponent 100111

Subtracting 100000 from exponent, we get 000111. The number is

$$0.111001101_2 \times 2^7 = 1110011.01_2 = 115.25_{10}$$

Example 2.25. What does the floating point number 0110100000010101 represent.

Solution : Mantissa is +0.110100000

Exponent is 010101

Subtracting 100000 from exponent, we get 110101.

The given number is $+ 0.110100000 \times 2^{-11} = + 0.00000000000110100000_2$

$$= + 0.000396728_{10}$$

The advantage of floating point representation is that very large and very small numbers can be easily expressed. Since the above representation uses 10 bit long mantissa, the accuracy in above representation is 9 bit (since 1 bit is used for sign). Fixed point 16 bit numbers are accurate to 15 bits. Thus breaking the 16 bit lengths into mantissa and exponent (to use floating point representation) reduces the accuracy to some extent. To ensure maximum accuracy the computers normalize the result of any floating point operation. In this process the most significant bit is placed next to the sign bit. Then the number is said to be normalized.

Example 2.26. Add the binary numbers 1 0 1 1 0 1 . 0 1 0 1 and 1 0 0 0 1 . 1 0 1.

Solution :

$$\begin{array}{rr} & 1\ 0\ 1\ 1\ 0\ 1\ .\ 0\ 1\ 0\ 1 \\ + & \quad 1\ 0\ 0\ 0\ 1\ .\ 1\ 0\ 1 \\ \hline & 1\ 1\ 1\ 1\ 1\ 0\ .\ 1\ 1\ 1\ 1 \end{array}$$

Example 2.27. Convert the binary number 1 1 0 0 1 . 0 0 1 0 1 1 to decimal.

Solution : The decimal equivalent is obtained as under

1	1	0	0	1	.	0	0	1	0	1	1	
2^4	2^3	2^2	2^1	2^0	.	2^{-1}	2^{-2}	2^{-3}	2^{-4}	2^{-5}	2^{-6}	weights

Decimal equivalent $= 1 \times 2^4 + 1 \times 2^3 + 1 \times 2^0 + 1 \times 2^{-3} + 1 \times 2^{-5} + 1 \times 2^{-6}$

$= 16 + 8 + 1 + 0.125 + 0.03125 + 0.015625 = 25.171875$

2.13. HEXADECIMAL NUMBERS

The hexadecimal number system is very convenient and extensively used because hexadec numbers are very short as compared to binary numbers.

Hexadecimal means 16. Thus the hexadecimal system has a base of 16. It uses 16 digits to repre all numbers. The digits are 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, *A, B, C, D, E, F.* Table 2.1 gives the binary-decin hexademical equivalence.

2.13.1. Hexadecimal to Binary Conversion

Convert each hexadecimal digit to its 4–bit binary equivalent using Table 2.1, *e.g.,* to convert 7 to the binary :

7	*B*	*F*	Hexadecimal number
↓	↓	↓	
0 1 1 1	1 0 1 1	1 1 1 1	Binary number

2.13.2. Binary to Hexadecimal Conversion

Convert each 4–bit binary into an equivalent hexadecimal using Table 2.1, *e.g.,*

		1 1 1 0	1 0 0 1	Binary	
		↓	↓		
		E	9	Hexadecimal	
and	1 1 1 1	1 1 0 1	1 0 0 0	0 1 1 0	Binary
	F	*D*	8	6	Hexadecimal

2.13.3. Hexadecimal to Decimal

One method to convert a hexadecimal number into its decimal equivalent is to first conver hexadecimal to binary and then convert binary to decimal.

A direct conversion of hexadecimal into decimal is also possible. Since the base of a hexadecimal is 16, the weights of different bits are 16^0, 16^1, 16^2, etc., starting with the bit on the extreme right. The decimal equivalent of a hexadecimal number equals the sum of all digits multiplied by their weights, *e.g.,*

E	7	*F*	6	Hexadecimal
16^3	16^2	16^1	16^0	Weights

$E7F6 = (E \times 16^3) + (7 \times 16^2) + (F \times 16^1) + (6 \times 16^0)$

$= (14 \times 4096) + (7 \times 256) + (15 \times 16) + (6 \times 1)$

$= 57344 + 1792 + 240 + 6 = 59382_{10}$

2.13.4. Decimal to Hexadecimal Conversion

One method is to convert the decimal to binary and then convert binary to hexadecimal (see Example 2.28 *a*).

The direct method is successive division by 16 and to write the hexadecimal equivalents of remainder, *e.g.,* to convert decimal number 5390 into hexadecimal.

16	5390	
16	336	remainder 14 : *E*
16	21	remainder 0 : 0
16	1	remainder 5 : 5
	0	remainder 1 : 1

The hexadecimal equivalent is 150*E*.

Example 2.28. Convert the hexadecimal number $8A3_{16}$ into decimal equivalent *(a)* by first converting into binary, *(b)* directly.

Solution : *(a)* Using Table 2.1 :

	8				*A*				3			Hexadecimal
1	0	0	0	1	0	1	0	0	0	1	1	Binary
2^{11}	2^{10}	2^9	2^8	2^7	2^6	2^5	2^4	2^3	2^2	2^1	2^0	Weights

The decimal number is $1 \times 2^{11} + 1 \times 2^7 + 1 \times 2^5 + 1 \times 2^1 + 1 \times 2^0$

$= 2048 + 128 + 32 + 2 + 1 = 2211_{10}$

(b)

8	*A*	3		Hexadecimal
16^2	16^1	16^0		Weights

The decimal number $= 8 \times 16^2 + 10 \times 16^1 + 3 \times 16^0 = 2048 + 160 + 3 = 2211_{10}$

Example 2.29. Convert the decimal number 268_{10} into hexadecimal, *(a)* by first converting it into binary, *(b)* directly.

Solution : *(a)*

2	268	
2	134	remainder 0
2	67	remainder 0
2	33	remainder 1
2	16	remainder 1
2	8	remainder 0
2	4	remainder 0
2	2	remainder 0
2	1	remainder 0
	0	remainder 1

The binary equivalent is 1 0000 1100$_2$

The hexadecimal equivalent is 1 0 *C* $= 10C_{16}$

(b)

16	268	
16	16	remainder 12 : *C*
16	1	remainder 0 : 0
2	1	remainder 1 : 1

Hexadecimal equivalent is 10 C.

Example 2.30. Convert decimal number 5741 into hexadecimal.

Solution :

16	5741		
16	358	remainder 13 :	D
16	22	remainder 6 :	6
16	1	remainder 6 :	6
	0	remainder 1 :	1

Hexadecimal number is $166D_{16}$.

Example 2.31. Convert hexadecimal number $D70_{16}$ into decimal equivalent.

Solution :

D	7	0	Hexadecimal
16^2	16^1	16^0	Weights

Decimal number = $13 \times 16^2 + 7 \times 16^1 = 3440_{10}$.

2.14. HEXADECIMAL ADDITION AND SUBTRACTION

Hexadecimal addtion can be performed in a number of ways. One method is to remember th equivalent decimal numbers of the hexadecimal digits and then perform the addition. Another is to conve the hexadecimal numbers into equivalent binary numbers and then adding (The arithmetic operations digital computer are performed on binary numbers only. The hexadecimal representation is used only f entering information into the computer). Still another and direct method is to use hexadecimal addition tab (Table 2.2).

Table 2.2. Hexadecimal addition table

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
1	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10
2	2	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11
3	3	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12
4	4	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13
5	5	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14
6	6	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15
7	7	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16
8	8	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17
9	9	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18
A	A	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19
B	B	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A
C	C	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B
D	D	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C
E	E	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D
F	F	10	11	12	13	14	15	16	17	18	19	1A	1B	1C	1D	1E

Table 2.2 can also be used for hexadecimal subtraction. For this purpose, find the smaller of the t numbers in the left hand column. Then move horizontally along the row till the higher of the two numb is found. The corresponding number in the top row is the result, *e.g.*, to find A–3, find 3 in the first colu and move along this row (containing 3) till A is found. The corresponding answer in the first row is 7.

Example 2.32. Add $AECF1_{16}$ and $15ACD_{16}$

Solution :

$$\begin{array}{r} A\ E\ C\ F\ 1 \\ +\ 1\ 5\ A\ C\ D \\ \hline C\ 4\ 7\ B\ E \end{array}$$

Example 2.33. Subtract $3A7_{16}$ from 1274_{16}

Solution :

$$\begin{array}{r} 1\ 2\ 7\ 4 \\ -\ \ \ 3\ A\ 7 \\ \hline E\ C\ D \end{array}$$

2.15. HEXADECIMAL MULTIPLICATION AND DIVISION

Hexadecimal multiplication and division requires the use of hexadecimal multiplication table (Table 2.3). Of course the multiplication and division can also be done by converting the numbers into binary numbers.

Table 2.3. Hexadecimal multiplication table

	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
1	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
2	0	2	4	6	8	A	C	E	10	12	14	16	18	1A	1C	1E
3	0	3	6	9	C	F	12	15	18	1B	1E	21	24	27	2A	2D
4	0	4	8	C	10	14	18	1C	20	24	28	2C	30	34	38	3C
5	0	5	A	F	14	19	1E	23	28	2D	32	37	3C	41	46	4B
6	0	6	C	12	18	1E	24	2A	30	36	3C	42	48	4E	54	5A
7	0	7	E	15	1C	23	2A	31	38	3F	46	4D	54	5B	62	69
8	0	8	10	18	20	28	30	38	40	48	50	58	60	68	70	78
9	0	9	12	1B	24	2D	36	3F	48	51	5A	63	6C	75	7E	87
A	0	A	14	1E	28	32	3C	46	50	5A	64	6E	78	82	8C	96
B	0	B	16	21	2C	37	42	4D	58	63	6E	79	84	8F	9A	A5
C	0	C	18	24	30	3C	48	54	60	6C	78	84	90	9C	A8	B4
D	0	D	1A	27	34	41	4E	5B	68	75	82	8F	9C	A9	B6	C3
E	0	E	1C	2A	38	46	54	62	70	7E	8C	9A	A8	B6	C4	D2
F	0	F	1E	2D	3C	4B	5A	69	78	87	96	A5	B4	C3	D2	E1

Example 2.34. Multiply 64_{16} to 19_{16}.

Solution :

$$\begin{array}{r} 6\ 4 \\ \times\ \ 1\ 9 \\ \hline 3\ 8\ 4 \\ 6\ 4\ \times \\ \hline 9\ C\ 4 \end{array}$$

Example 2.35. Divide $1EC87_{16}$ by $A5_{16}$.

Solution :

$$\begin{array}{r} A5\ )\ 1\ E\ 7\ 8\ 7\ (\ 2FC \quad \text{quotient} \\ 1\ 4\ A \\ \hline A\ 2\ 8 \\ 9\ A\ B \\ \hline 7\ 8\ 7 \\ 7\ B\ C \\ \hline 1\ B \quad \text{remainder.} \end{array}$$

2.16. OCTAL NUMBER SYSTEM

The octal number system has a base of 8. It uses eight digits, *i.e.*, 0, 1, 2, 3, 4, 5, 6, 7. It was very commonly used in early mini computers. However, the present day computer systems use hexadecimal numbers. Table 2.1 gives the binary–decimal–hexadecimal–octal equivalent.

2.16.1. Octal to Decimal Conversion

Each successive digit position, in octal system, is an increasing power of 8, beginning with the right most column with 8^0. Conversion of octal number into equivalent decimal number is obtained by multiplying each digit by its weight and summing the products.

1	0	5	4	Octal number
8^3	8^2	8^1	8^0	Weights

$$1054_8 = (1 \times 8^3) + (0 \times 8^2) + (5 \times 8^1) + (4 \times 8^0) = 512 + 0 + 40 + 4 = 556_{10}$$

2.16.2. Decimal to Octal Conversion

It is done through successive division by 8 and by writing the remainders. The remainders when read upwards give the octal number, *e.g.*, to convert 574_{10} into octal number.

8	574	
8	71	remainder 6
8	8	remainder 7
8	1	remainder 0
	0	remainder 1

Thus $574_{10} = 1076_8$

2.16.3. Octal to Binary Conversion

Each octal digit represents 3 binary digits. To convert an octal number to binary number, each octal digit is replaced by its 3 digit binary equivalent, *e.g.*,

7	1	Octal number
↓	↓	
111	001	Binary number

Thus $71_8 = 111001_2$

2.16.4. Binary to Octal Conversion

Replace each 3 bit binary by its octal equivalent, *e.g.*,

1 1 0	1 0 1	Binary number
↓	↓	
6	5	Octal number

Thus $110101_2 = 65_8$

Example 2.36. Convert the following binary numbers into equivalent octal numbers.

(a) 1001101.1011 *(b)* 111000101

Solution : *(a)*

0 0 1	0 0 1	1 0 1	.	1 0 1	1 0 0
↓	↓	↓		↓	↓
1	1	5		5	4

Thus $1001101.1011_2 = 115.54_8$

(b)

$$\underbrace{1\,1\,1}_{\downarrow}\quad\underbrace{0\,0\,0}_{\downarrow}\quad\underbrace{1\,0\,1}_{\downarrow}$$

$$7 \qquad 0 \qquad 5$$

Thus $111000101_2 = 705_8$

Example 2.37. Convert the following octal numbers to binary *(a)* 24_8 *(b)* 160_8.

Solution : *(a)*

$$2 \qquad 4_8$$
$$\downarrow \qquad \downarrow$$
$$010 \qquad 100$$

Thus $24_8 = 010100_2$

(b)

$$1 \qquad 6 \qquad 0_8$$
$$\downarrow \qquad \downarrow \qquad \downarrow$$
$$001 \qquad 110 \qquad 000$$

Thus $160_8 = 001110000_2$

Example 2.38. *(a)* Convert the octal number 1502_8 into decimal number.

(b) Convert decimal number 468 into octal number.

Solution : *(a)*

1	5	0	2	Octal number
8^3	8^2	8^1	8^0	Weights

Thus $1502_8 = 1 \times 8^3 + 5 \times 8^2 + 0 \times 8^1 + 2 \times 8^0 = 834_{10}$

(b)

8	468	
8	58	remainder 4
8	7	remainder 2
	0	remainder 7

Thus $468_{10} = 724_8$

2.17. BINARY CODED DECIMAL (BCD)

Computers work with binary numbers. We work with decimal numbers. A code is needed to represent decimal numbers as binary numbers.

A weighted binary code is one in which each number carries a certain weight. A string of 4 bits is known as nibble. Binary coded decimal (BCD) means that each decimal digit is represented by a nibble (binary code of 4 digits). Many BCD codes have been proposed, *e.g.*, 8421, 2421, 5211, X53. Out of these 8421 code is the most predominant BCD code. The designation 8421 indicates the weights of the 4 bits (8, 4, 2 and 1 respectively starting from the left most bit). When one refers to a BCD code, it always means 8421 code. Though 16 numbers (2^4) can be represented by 4 bits, only 10 of these are used. Table 2.4 shows the BCD code. The remaining 6 combinations, *i.e.*, 1010, 1011, 1100, 1101, 1110 and 1111 are invalid in 8421 BCD code. To express any number in BCD code, each decimal number is replaced by the appropriate four bit code of Table 2.4. BCD code is used in pocket calculators, electronic counters, digital voltmeters, digital clocks etc. Early versions of computers also used BCD code. However, the BCD code was discarded for computers because this code is slow and more complicated than binary. Table 2.5 shows some decimal numbers and their representation in octal, hexadecimal, binary and BCD systems.

Table 2.4. 8421 BCD code

Decimal	8421 BCD
0	0000
1	0001
2	0010
3	0011
4	0100
5	0101
6	0110
7	0111
8	1000
9	1001

Table 2.5. Number systems

Decimal	Octal	Hexadecimal	Binary	8421 BCD
0	0	0	0000 0000	0000 0000 0000
1	1	1	0000 0001	0000 0000 0001
2	2	2	0000 0010	0000 0000 0010
3	3	3	0000 0011	0000 0000 0011
4	4	4	0000 0100	0000 0000 0100
5	5	5	0000 0101	0000 0000 0101
6	6	6	0000 0110	0000 0000 0110
7	7	7	0000 0111	0000 0000 0111
8	10	8	0000 1000	0000 0000 1000
9	11	9	0000 1001	0000 0000 1001
10	12	A	0000 1010	0000 0001 0000
11	13	B	0000 1011	0000 0001 0001
12	14	C	0000 1100	0000 0001 0010
13	15	D	0000 1101	0000 0001 0011
14	16	E	0000 1110	0000 0001 0100
15	17	F	0000 1111	0000 0001 0101
16	20	10	0001 0000	0000 0001 0110
32	40	20	0010 0000	0000 0011 0010
64	100	40	0100 0000	0000 0110 0000
128	200	80	1000 0000	0001 0010 1000
200	310	C8	1100 1000	0010 0000 0000
255	377	FF	1111 1111	0010 0101 0101

Example 2.39. Express the following decimal numbers in 8421 BCD code : *(a)* 90 *(b)* 115 *(c)* 410 *(d)* 12.1 *(e)* 38.2 *(f)* 1507

Solution : *(a)*

9	0
↓	↓
1001	0000

(b)

1	1	5
↓	↓	↓
0001	0001	0101

(c)

4	1	0
↓	↓	↓
0100	0001	0000

(d)

1	2	.	1
↓	↓		↓
0001	0010	.	0001

(e)

3	8	.	2
↓	↓		↓
0011	1000	.	0010

(f)

1	5	0	7
↓	↓	↓	↓
0001	0101	0000	0111